

АРВНПР-ТЗ

Павел Попов

17 марта 2025 г.

1 Постановка задачи

1.1 Общая постановка

Пусть имеется большой дата-центр (серверная), содержащая однотипные устройства X . Необходимо производить мониторинг работоспособности серверной, основываясь на исходящих от устройств пакетов унифицированного формата. Реализовать систему в микросервисной архитектуре, содержащую:

- API, реализованном с помощью Python-фреймворка FastAPI;
- База данных - MongoDB;
- Балансировщик нагрузки - Nginx;
- Брокер сообщений - RabbitMQ;
- Кэш - Redis;
- ELK-стек Elasticsearch, Logstash и Kibana, + Filebeat для сбора и анализа логов;
- Мониторинг Grafana и Prometheus.
- Google Protobuf3 - для установки стандарта обмена сообщениями.

1.2 Уточнения к постановке задачи

Естественно заметить, что разрабатываемое решение представляет собой систему обработки и/или логической сортировки пакетов-сообщений, циркулируемых по компонентам.

Логично установить следующий унифицированный формат пакета.

```
message Batch {  
    int32 device_id = 1;  
    float alpha = 2;  
    float beta = 3;  
    string timestamp = 4;  
}
```

Пакет Batch представляет собой *protobuf*-сообщение. Библиотека *protobuf* предоставляет удобные инструменты для сериализации/десериализации сообщений.

- `device_id` - эмуляция id устройства, с которого было отправлено сообщение.
- `alpha`, `beta` - логические абстрактные поля, с возможностью переопределения в будущем (например, можно представить, что собираем данные с устройств: температуру, загрузку процессора, количество занятой оперативной памяти и пр.), на основании которых в дальнейшем будет определяться путь пакетов в решении.
- `timestamp` - время отправки данного пакета.

1.3 Бизнес-логика решения

Остановимся подробнее на каждом из компонентов: Data Simulator, IoT Controller и Rule Engine и поясним их логику обработки пакетов Batch.

- Data Simulator (будет заменён на Tsung) генерирует данные. Поддерживаются настройки:
 - Количество клиентов
 - Частота отправки сообщений в секунду.
- IoT Controllers осуществляют валидацию пакетов по следующему алгоритму принятия решений:
 - Если поле `alpha` входящего пакета < 25 , то IoT Controller не принимает пакет.

- Если $\alpha \geq 25$, то пакет принимается, сохраняется в БД и передается в Rule Engine при помощи RabbitMQ.
- Rule Engine выполняет data-processing и заполняет БД алёртами о событиях instant rule, ongoing rule.
Ведёт учёт последних $k = 10$ пакетов при помощи БД.
 - Если поле $\alpha < 50$, то срабатывает instant rule и выполняется запись в БД с временной меткой этого события.
 - Для каждого из подключённых клиентов создаётся "микро стек" в БД размер которого всегда не превышает 10 сообщений. Каждое новое сообщение попадает в этот стек.
 - Отслеживается следующее: если поле $\beta \geq 75$ встречается более 5 раз на протяжении 10 пакетов для данного device id, то срабатывает ongoing rule, и запись с уведомлением помещается в БД.
- Балансировщик нагрузки Nginx
 - Проксирует запросы с эмулятора нагрузки на приёмники запросов от устройств (Controllers), а также отвечает за распределение нагрузки.
- Redis - за последние 30 секунд для каждого контроллера хранит метрики:
 - Средние α, β за последние k секунд, количество принятых и отклонённых пакетов.

1.4 Формирование требований

Предлагается сформулировать перечень требования к разрабатываемому решению. Необходимо выполнить решение согласно требованиям, а затем постараться улучшить полученные результаты, чтобы превзойти поставленные поставленные задачи по производительности.

1.4.1 Максимальное количество устройств

На данный момент было протестировано для одного Controller и одного Data Simulator - 460 устройств-отправителей, с эмуляцией данных по одному пакету в секунду со скоростью обработки 0.3 секунды на запрос. Предлагается увеличивать количество устройств отправителей и количество контроллеров, масштабируя нагрузку при помощи Nginx. Ожидается прямо пропорциональное повышение производительности.

1.4.2 Максимально возможное количество устройств одновременно

Ожидается увеличить пропускную способность до 2000 пакетов в секунду без существенной потери обработки запроса (до 1 секунды на запрос), за счёт увеличения количества Controllers.

1.4.3 Время обработки запросов на уровне базы данных

Использовать асинхронные операции с БД. Предлагается считать производительным решение со скоростью CRUD-операций не дольше 1 секунды.

1.4.4 Объем хранимых данных

Предлагается разделить данные на оперативные и длительного хранения. Оперативные данные хранить на протяжении последней недели, а данные длительного хранения (из кэша Redis) хранить в течение 365 дней. Итого 20 гб на оперативные данные и 1 гб на данные длительного хранения (небольшие отчёты).